

Deep Learning for Master Students

Mandatory Coding Task

Name: Luca Martin -- **Matriculation Number:** 3757121

This report documents the coding task for the module “Deep Learning for Master Students” - winter semester 2025/26. It includes a setup guide ([Setup guide](#)) explaining how to configure the programming environment and which parameters need to be set execute the submitted code. Additionally, this document shows, which files to run and presents the training and testing results ([Task 2 - EuroSAT RGB](#) and [Task 3 - EuroSAT MS](#)), including validation performance graphs, final test accuracy and true positive rate (TPR) per class, and the top-5 and bottom-5 images for three selected classes.

Each model was trained on 10.000 training images, 5.000 validation images and tested on 5.000 test images. The training was run with a batchsize of 16 in 625 batches for 20 epochs with a learning rate of 0.0001.

Setup guide

1. Create and activate a virtual environment inside the projekt-root folder:

```
python3 -m venv venv
source venv/bin/activate
```

2. Install the requirements:

```
pip install -r requirements.txt
```

3. Unzip the datasets into:

```
your_project_root/
├── coding_task_data/
│   ├── EuroSAT_RGB/
│   └── EuroSAT_MS/
```

4. Open the `config.yml` and set the `project_root` and the `dataset_root`. If you unzipped the datasets exactly as described above, no changes should be necessary; otherwise, adjust the paths accordingly:

```
paths:
  project_root: "."

  # For task 2:
  dataset_root: "./coding_task_data/EuroSAT_RGB" # or "[your_dataset_root]"

  # For task 3: Important: For the code to run properly, the dataset path has to end on "_MS"!
  dataset_root: "./coding_task_data/EuroSAT_MS" # or "[your_dataset_root]_MS"
```

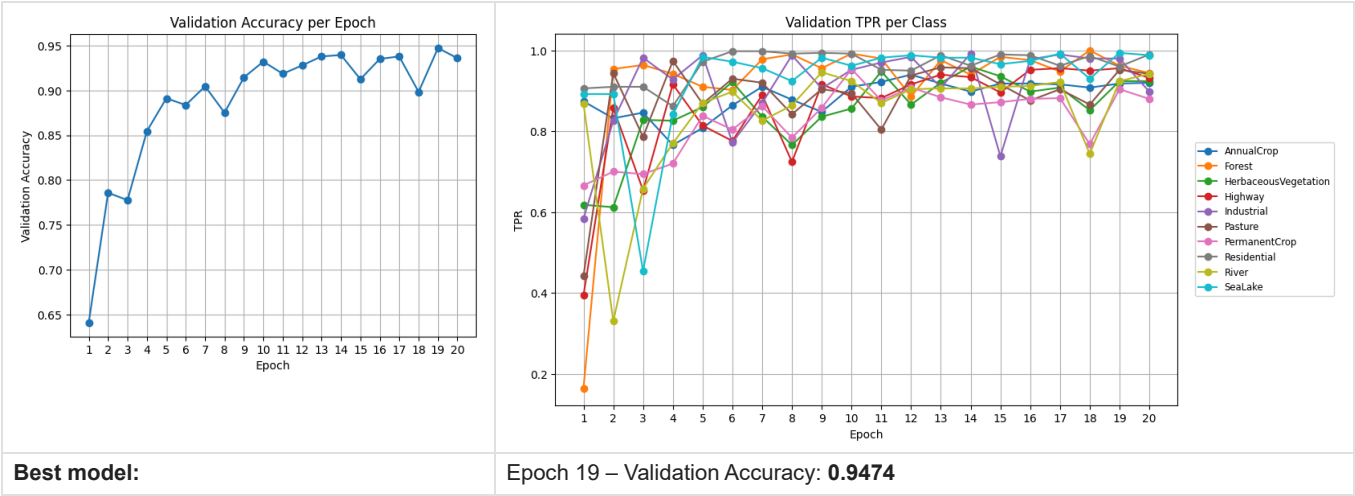
Task 2 - EuroSAT RGB

- Set `dataset_root: "./coding_task_data/EuroSAT_RGB"` inside the `config.yml` (assuming, you saved the datasets as described above)

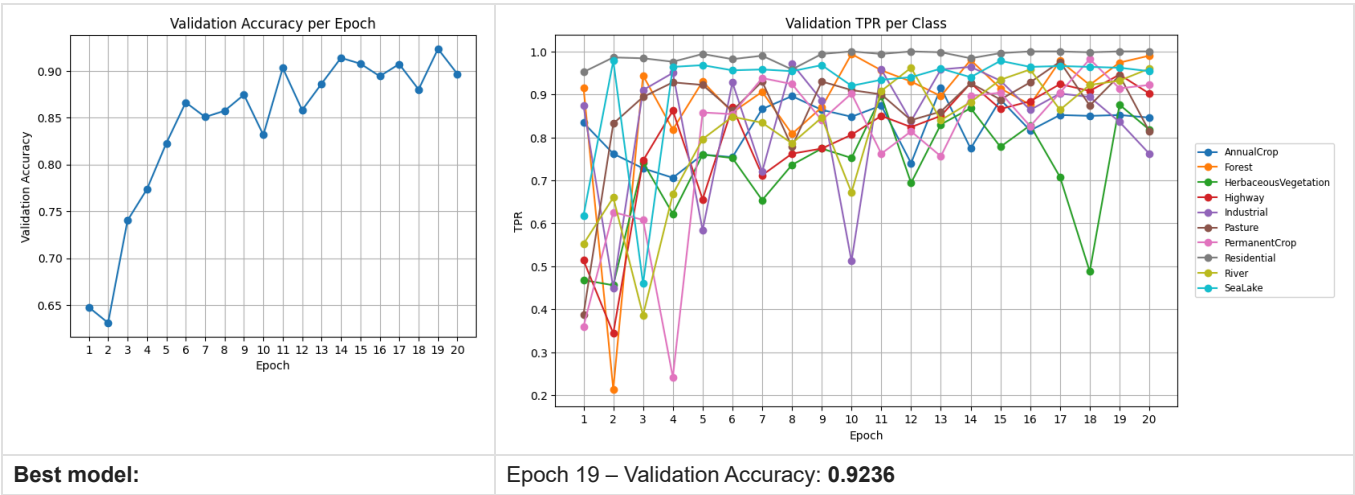
Note: You can either run the prediction and reproduction routine on the provided `best_model.pt` (start at step "3. run the prediction") or retrain the model and then run the prediction and reproduction (start at step "1. create disjoint splits")

1. To **create disjoint train / val / test splits**, run `data_splitter.py`
2. To **train the model**, run `./train.py`. This overrides the provided `model.pt` files
3. To **run the prediction** on the test data using the final model, run `./predict.py`. This either uses the provided `final_model.pt` file or (in case you trained the model again) the newly created one and saves the test logits every time.
4. To **run the reproduction routine** using the final model, run `./reproduce.py`. To save the logits in the initial run (if you have not run `predict.py` or want to save them again), set `save_logits: True` inside the `config.yml`. To compare the logits, set `save_logits: False`.

Results - Mild data augmentation



Results - Strong data augmentation

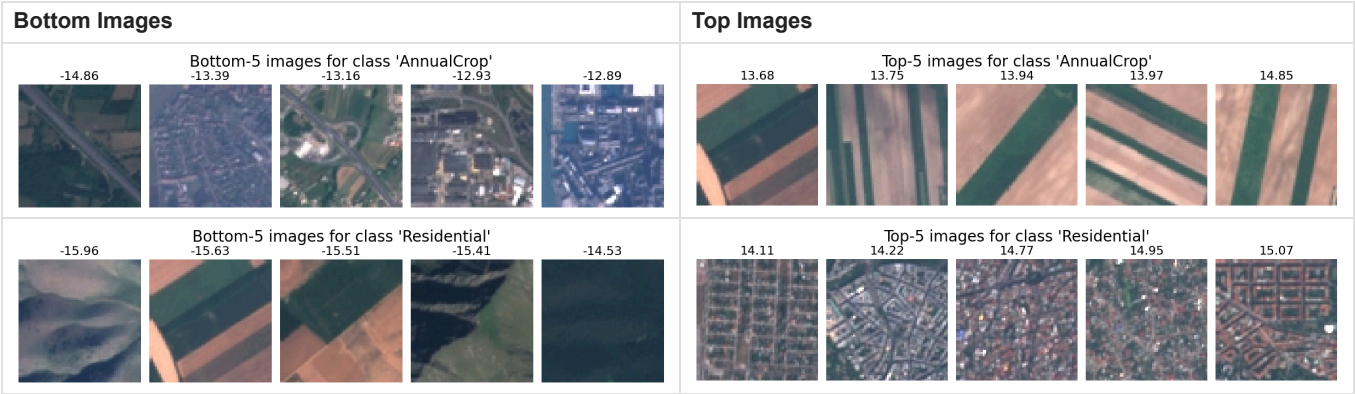


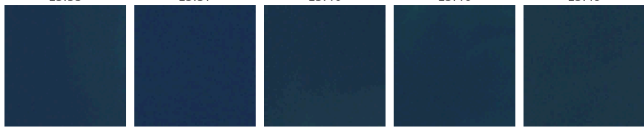
Finally selected model = Mild Data Augmentation

- Test Accuracy: 0.9336
- Test TPR per class:

0: Annual Crop	1: Forest	2: Herbaceous Vegetation	3: Highway	4: Industrial	5: Pasture	6: Permanent Crop	7: Residential	8: River	9: Sea Lake
0.942	0.938	0.912	0.93	0.88	0.9	0.898	0.996	0.952	0.988

Ranking

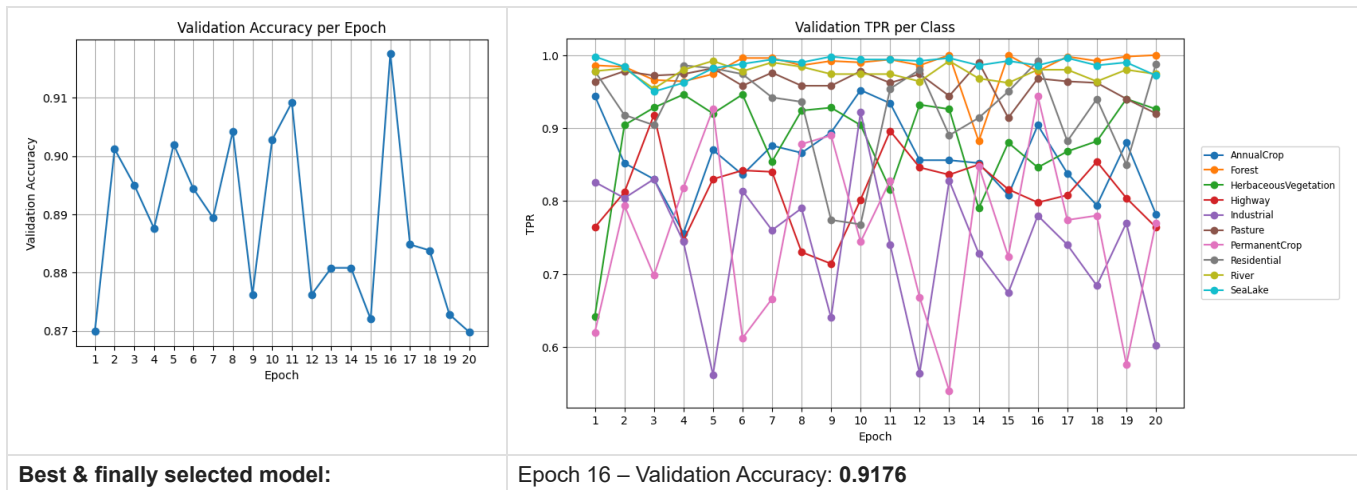


Bottom Images	Top Images
Bottom-5 images for class 'SeaLake' 	Top-5 images for class 'SeaLake' 

Task 3 - EuroSAT MS

- The ToDo's for Task 3 are almost identical to Task 2, except for the dataset path: `dataset_root: "./coding_task_data/EuroSAT_MS"`
- The path must end with `"_MS"` for the code to execute properly and process the multispectral correctly.
- Then run: `(data_splitter.py, train.py, predict.py, reproduce.py)` as described above.

Results & finally selected model

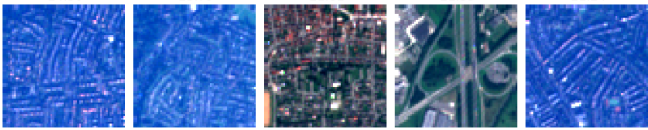
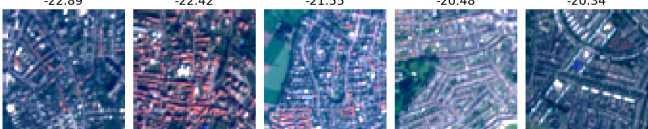


- Test Accuracy:** 0.8698
- Test TPR per class:**

0: Annual Crop	1: Forest	2: Herbaceous Vegetation	3: Highway	4: Industrial	5: Pasture	6: Permanent Crop	7: Residential	8: River	9: Sea Lake
0.784	1.0	0.92	0.784	0.58	0.91	0.768	0.988	0.982	0.982

Ranking

To display these images, they were converted from multispectral (MS) to RGB. Any color inaccuracies are due to this conversion.

Bottom Images	Top Images
Bottom-5 images for class 'AnnualCrop' 	Top-5 images for class 'AnnualCrop' 
Bottom-5 images for class 'Residential' 	Top-5 images for class 'Residential' 
Bottom-5 images for class 'SeaLake' 	Top-5 images for class 'SeaLake' 